

Online Supplemental Material for “Urban Delivery with a Coordinated Fleet of Trucks and Drones”

C. Proofs

C.1. Proof of Proposition 1.

If we relax constraint (5) for customer j ($j \in \mathcal{S} \setminus \mathcal{P}_s$), we can obtain the minimum cost of all customers in $\mathcal{P}_s$ being served by their assigned trucks or being rejected (if the assigned truck is $\emptyset$) by solving the proposed model, which is a lower bound of the generalized cost. Based on the father node f , the route cost of a truck k that is not assigned a new customer remains φ_k^f , while the route cost of a truck κ that is assigned a new customer changes to φ_κ^s . Given that the travel cost and soft time-window penalty are always positive, φ_κ^s must be greater than φ_κ^f because truck κ serves an extra customer. Therefore, $\sum_{k \in \mathcal{K}, k \neq \kappa} \varphi_k^f + \varphi_\kappa^s$ is a valid lower bound. $\square$

C.2. Proof of Proposition 2.

A valid UB at node s can be obtained by constructing a feasible solution based on the LB at that node. Specifically, if we reject all remaining new customers who have not been assigned to trucks at node s , we can obtain a feasible solution $\mathcal{S} \setminus \mathcal{P}$, which generates a UB as $LB + \sum_{j \in \mathcal{S} \setminus \mathcal{P}_s} F_j^R$. $\square$

C.3. Proof of Proposition 3.

At each branch, we select a new customer i and assign it to at most $|\mathcal{K}|$ trucks or reject it, which means $|\mathcal{K}| + 1$ child nodes are generated. Later, when all new customers in $\mathcal{S}$ are assigned, $\mathcal{S} \setminus \mathcal{P} = \emptyset$, there is no new customer to be served. The UB of this node is exactly the optimal solution of this node, which equals the LB . Therefore, the node is the leaf node, and the maximum depth of the BB-tree is $|\mathcal{S}|$. The maximum number of nodes in the BB-tree is $\sum_{n=0}^{|\mathcal{S}|} (|\mathcal{K}| + 1)^n = \frac{(|\mathcal{K}|+1)^{|\mathcal{S}|+1} - 1}{|\mathcal{K}|}$. $\square$

C.4. Proof of Proposition 4.

We have three groups of constraints in the subproblem SP : (i) the first group (Constraints (D.2)-(D.9)) ensures that the truck visits every node within its hard time window, which is also required in RSP (Constraints (D.21)-(D.28)); (ii) the second group (Constraints (D.10)-(D.11)) requests that the trucks pick up parcels before serving the corresponding customers, which is defined in the graph structure of RSP and restricted by constraints (D.25) and (D.29); and (iii) the third group (Constraints (D.12)-(D.14)) routes drones to meet and supply trucks, which is reflected in the meeting arcs of the graph. The objective function of both the RSP and SP is to minimize the cost and soft time-window penalty, with the RSP incorporating drone travel costs through meeting arcs. Therefore, the two models have the same solution. $\square$

C.5. Proof of Proposition 5.

The feasibility of the constructed time schedule is guaranteed by the following two properties: First, the label feasibility conditions in Section 4.2.1 ensure that for every node j in the path, there exists at least one time point τ_j that satisfies $W_j' \leq \tau_j \leq U_j'$. Second, the greedy construction method preserves feasibility as follows. (i) For each node j , setting $\tau_j = \max(W_j, \tau_i + T_{ij})$ ensures no early service. (ii) When $\tau_j > U_j'$, enforcing $\tau_j = U_j'$ guarantees that node j will not violate its latest allowable delivery time U_j' . (iii)

The recursive adjustment of predecessors cannot violate their hard time-window constraints, because any such label has been removed based on the feasible label definition (Section 4.2.1). Thus, the algorithm either directly finds a feasible schedule through forward propagation or corrects violations via backward adjustment while preserving W'_i bounds. $\square$

C.6. Proof of Proposition 6.

Based on the initial time schedule, nodes on the path can be divided into three types: (i) nodes that are exactly at their latest delivery time U'_i , which will be infeasible if moving backward; (ii) nodes that lie between $[W_i, U_i)$, which will either maintain or increase the delay penalties if moving backward; and (iii) nodes that lie between $[W'_i, W_i)$. Moving these kinds of nodes backward will reduce the early penalties, and such an operation is not allowed. According to our construction method, it is only when a node j that violates its hard time window moves forward to U'_j that a predecessor node i can ultimately fall into $[W'_i, W_i)$. Therefore, if we move a node located between $[W'_i, W_i)$ backward, there must be a node j being moved beyond U'_j , which will lead to infeasibility. Overall, none of these three nodes can benefit from moving backward. $\square$

C.7. Proof of Proposition 7.

For a segment with an equal number of delay nodes and early nodes, if we move the segment backward, the delay node number will increase with the early node number decreasing, resulting in $|\mathcal{N}^{delay}| > |\mathcal{N}^{early}|$. In this case, moving the segment back will bring $|\mathcal{N}^{delay}|$ additional delay penalties and reduce $|\mathcal{N}^{early}|$ early penalties, which will increase the overall penalties when $|\mathcal{N}^{delay}| > |\mathcal{N}^{early}|$. The same is true when we continue to move the segment forward, which will lead to an increase in early penalties greater than the reduction in delay penalties. $\square$

C.8. Proof of Proposition 8.

We establish the correctness of Algorithm 2 in two steps:

1. Completeness of Candidate Movements: The algorithm exhaustively evaluates all critical time points at which the delay cost structure may change: (i) Movement to U_i (Line 6): When node i reaches its latest penalty-free delivery time U_i , it ceases to be a delay node (n^{delay} decreases). (ii) Movement to W_i (Line 10): When node i moves forward beyond its earliest penalty-free delivery time W_i , it becomes an early node (n^{early} increases). (iii) Movement to W'_i (Line 21): When node i moves to its earliest allowable delivery time W'_i , further movement is infeasible. Crucially, within any interval between these critical points, the numbers of delay and early nodes are constant, making the objective function linear. Therefore, optimality must occur at one of these thresholds. The loop (Lines 2–30) guarantees that all nodes' critical points are evaluated.

2. Optimality of Termination Condition: By Proposition 7, the optimality criterion $n^{early} = n^{delay}$ ensures that no further improvement is possible. The recorded *optimal_movement* (Line 32) is thus the optimal solution. $\square$

C.9. Proof of Proposition 9.

First, the initial time-schedule construction procedure sets an initial service time for each node, the complexity of which is $O(|\mathcal{N}|)$. Second, after dividing the path into ζ segments, we calculate the optimal movement for each segment. This requires a total complexity of $O(|\mathcal{N}|)$, since every node in every segment needs to be processed once. Next, for the segments that have conflicts, we combine them and recalculate their optimal movement as a whole. For ζ segments, the maximum number of conflicts that can occur is $\zeta - 1$; for each conflict, the number of nodes that need to be reprocessed is at most $|\mathcal{N}|$. Therefore, the complexity of conflict identification and processing can be represented as $O((\zeta - 1)|\mathcal{N}|)$. Overall, the complexity of the strategy is $O((\zeta + 1)|\mathcal{N}|)$. Because a segment consists of at least two nodes (a delay node and a non-delay node), a path containing $|\mathcal{N}|$ nodes can be divided into up to $|\mathcal{N}|/2$ segments. Therefore, the complexity can be written as $O\left(\left(\frac{|\mathcal{N}|}{2} + 1\right)|\mathcal{N}|\right)$, which is solvable in quadratic time—i.e., $O(n^2)$ ($n = |\mathcal{N}|$). $\square$

C.10. Proof of Proposition 1.

Given two partial paths $\mathcal{B}(L_1)$ and $\mathcal{B}(L_2)$, L_1 dominates L_2 if and only if the former has a lower or equal objective value with fewer nodes to be extended (Li 2010). $t(L_1) \leq t(L_2)$ and $\mathcal{V}(L_2) \subseteq \mathcal{V}(L_1)$ indicate that the nodes that still need to be extended by L_2 are either already reached by L_1 or can be reached by L_1 with the same schedule as L_2 (Fachini and Armentano 2020). $\square$

D. Models

D.1. Subproblem Model

$$SP: \min \sum_{(i,j) \in \mathcal{A}} F_{ij}^K x_{ij} + \sum_{i \in \mathcal{C} \cup \mathcal{S}} 2F_{oi}^D y_i + F^{TW} \sum_{i \in \mathcal{C} \cup \mathcal{S}} ((\tau_i - U_i)^+ + (W_i - \tau_i)^+) \quad (D.1)$$

$$s.t. \sum_{(i,j) \in \mathcal{A}: i=v_k} x_{i,j} - \sum_{(j,i) \in \mathcal{A}: i=o} x_{j,i} = 0 \quad (D.2)$$

$$\sum_{(i,j) \in \mathcal{A}} x_{ij} - \sum_{(j,i) \in \mathcal{A}} x_{ji} = 0, \quad \forall i \in \mathcal{C} \cup \mathcal{S} \quad (D.3)$$

$$\sum_{(i,j) \in \mathcal{A}} x_{ij} = 1, \quad \forall i \in \mathcal{C} \quad (D.4)$$

$$\sum_{(j,n) \in \mathcal{A}} x_{jn} = 1, \quad \forall j \in \mathcal{S} \quad (D.5)$$

$$M(x_{ij} - 1) + \tau_i + T_{ij}^K \leq \tau_j, \quad \forall (i,j) \in \mathcal{A} \quad (D.6)$$

$$W'_i \leq \tau_i \leq U'_i, \quad \forall i \in \mathcal{C} \cup \mathcal{S} \quad (D.7)$$

$$\tau_{v_k} \geq R_k, \quad (D.8)$$

$$\tau_o \leq P_k, \quad (D.9)$$

$$M(z_i^j - 1) + T_{oi}^D \leq \tau_i, \quad \forall i \in \mathcal{C} \cup \mathcal{S}, \forall j \in \mathcal{S} \quad (D.10)$$

$$M(z_i^j - 1) + \tau_i \leq \tau_j, \quad \forall i \in \mathcal{C} \cup \mathcal{S}, \forall j \in \mathcal{S} \quad (D.11)$$

$$2T_{oi}^D z_i^j \leq E^D, \quad \forall i \in \mathcal{C} \cup \mathcal{S}, \forall j \in \mathcal{S} \quad (D.12)$$

$$\sum_{j \in \mathcal{S}} z_i^j \leq L^D, \quad \forall i \in \mathcal{C} \cup \mathcal{S} \quad (D.13)$$

$$z_i^j \leq y_i, \quad \forall i \in \mathcal{C} \cup \mathcal{S}, \forall j \in \mathcal{S} \quad (\text{D.14})$$

$$\sum_{i \in \mathcal{C} \cup \mathcal{S}} y_i \leq 1, \quad (\text{D.15})$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in \mathcal{A} \quad (\text{D.16})$$

$$y_i \in \{0, 1\}, \quad \forall i \in \mathcal{C} \cup \mathcal{S} \quad (\text{D.17})$$

$$z_i^j \in \{0, 1\}, \quad \forall i \in \mathcal{C} \cup \mathcal{S}, \forall j \in \mathcal{S} \quad (\text{D.18})$$

$$\tau_i \geq 0, \quad \forall i \in \mathcal{N} \quad (\text{D.19})$$

D.2. Reformulated Subproblem Model

$$RSP: \min \sum_{(i,j) \in \mathcal{A}^N} F_{ij}^K x_{ij} + \sum_{(i,j) \in \mathcal{A}^M} (F_{ij}^K + 2F_{oj}^D) x_{ij} + F^{TW} \sum_{i \in \mathcal{C} \cup \mathcal{C}' \cup \mathcal{S}} ((\tau_i - U_i)^+ + (W_i - \tau_i)^+) \quad (\text{D.20})$$

$$s.t. \sum_{(v_k,j) \in \mathcal{A}'} x_{v_k,j} - \sum_{(j,o) \in \mathcal{A}'} x_{j,o} = 0 \quad (\text{D.21})$$

$$\sum_{(i,j) \in \mathcal{A}'} x_{ij} - \sum_{(j,i) \in \mathcal{A}'} x_{ji} = 0, \quad \forall i \in \mathcal{C} \cup \mathcal{C}' \cup \mathcal{S} \quad (\text{D.22})$$

$$\sum_{(i,j) \in \mathcal{A}'} x_{ij} + \sum_{(\iota_i,j) \in \mathcal{A}'} x_{\iota_i j} = 1, \quad \forall i \in \mathcal{C} \quad (\text{D.23})$$

$$\sum_{(i,j) \in \mathcal{A}'} x_{ij} = 1, \quad \forall i \in \mathcal{S} \quad (\text{D.24})$$

$$M(x_{ij} - 1) + \tau_i + T_{ij}^K \leq \tau_j, \quad \forall (i, j) \in \mathcal{A}' \quad (\text{D.25})$$

$$W_i' \leq \tau_i \leq U_i' \quad \forall i \in \mathcal{C} \cup \mathcal{C}' \cup \mathcal{S} \quad (\text{D.26})$$

$$\tau_{v_k} \geq R_k \quad (\text{D.27})$$

$$\tau_o \leq P_k \quad (\text{D.28})$$

$$M(x_{ij} - 1) + T_{oj}^D \leq \tau_j, \quad \forall (i, j) \in \mathcal{A}^M \quad (\text{D.29})$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in \mathcal{A}' \quad (\text{D.30})$$

$$\tau_i \geq 0, \quad \forall i \in \mathcal{N}' \quad (\text{D.31})$$

Objective function (D.20) is to minimize the generalized cost. Constraints (D.21) and (D.22) are the flow conservation constraints. Constraints (D.23) and (D.24) ensure that the order assigned to this truck is served. Constraints (D.25)-(D.29) restrict the time at which each truck reaches each node. Constraints (D.30)-(D.31) define the ranges of decision variables.

D.3. Offline Model

We introduce two new variables:

- a_{ij}^d represents the visit order of the meeting points of drone d . Specifically, a_{oi}^d equals 1 indicates that the first meeting point visited by drone d is node i , and a_{ij}^d equals 1 if i and j are two consecutive meeting points sequentially visited by drone d .

- μ_i^d represents the time at which drone d visits meeting point i .

Building upon the original model proposed in Section 3.2, we relax constraints (17)-(18) and replace constraint (11) by constraints (D.32)–(D.39) to construct an offline model under full knowledge of all orders' locations, time windows, and release times. We denote the release time of order i as T_i^R .

$$\sum_{k \in \mathcal{K}} y_i^k = \sum_{d \in \mathcal{D}} \sum_{j \in \mathcal{N}} a_{ji}^d, \quad \forall i \in \mathcal{C} \cup \mathcal{S} \quad (\text{D.32})$$

$$\sum_{i \in \mathcal{N}} a_{ij}^d = \sum_{i \in \mathcal{N}} a_{ji}^d, \quad \forall j \in \mathcal{N}, \forall d \in \mathcal{D} \quad (\text{D.33})$$

$$\sum_{d \in \mathcal{D}} \sum_{i \in \mathcal{N}} a_{oi}^d \leq |\mathcal{D}|, \quad (\text{D.34})$$

$$M(z_i^{kj} - 1) + T_{oi}^D + T_j^R \leq \mu_i^d \quad \forall i \in \mathcal{C}_k \cup \mathcal{S}, \forall j \in \mathcal{S}, \forall k \in \mathcal{K}, \forall d \in \mathcal{D} \quad (\text{D.35})$$

$$\mu_i^d \leq \tau_i^k, \quad \forall i \in \mathcal{C}_k \cup \mathcal{S}, \forall k \in \mathcal{K}, \forall d \in \mathcal{D} \quad (\text{D.36})$$

$$M(a_{ij}^d - 1) + \mu_i^d + T_{io}^D + T_{oj}^D \leq \mu_j^d, \quad \forall i \in \mathcal{C}_k \cup \mathcal{S}, \forall j \in \mathcal{C}_k \cup \mathcal{S}, \forall d \in \mathcal{D} \quad (\text{D.37})$$

$$a_{ij}^d \in \{0, 1\}, \quad \forall i, j \in \mathcal{N}, \forall d \in \mathcal{D} \quad (\text{D.38})$$

$$\mu_i^d \geq 0, \quad \forall i \in \mathcal{N}, \forall d \in \mathcal{D} \quad (\text{D.39})$$

Constraint (D.32) ensures that all nodes designated as meeting points are visited by drones. Constraint (D.33) enforces flow balance for drone routes. Constraint (D.34) limits the total number of drones. Constraints (D.35)–(D.37) govern the visiting time of drone to each meeting point. The last two constraints define the domain of the new decision variables.

E. Examples

EXAMPLE 1. Figure E.1(a) shows the initial routes for 3 trucks, in which the solid line represents the completed route—which is nonadjustable—and the dashed line is the incomplete part, which can be adjusted. Figure E.1(b) shows meeting points (i.e., at nodes 3 and 9) for the resupply to 2 trucks with 2 new orders (i.e., with delivery to nodes 8 and 9) and the corresponding adjusted routes at one decision epoch. Figure E.1(c) shows meetings (i.e., at nodes 10 and 11) for the resupply to 2 trucks with another 2 new orders (i.e., with delivery directly to nodes 10 and 11) and the corresponding routes at the next decision epoch.

EXAMPLE 2. Figure E.2 gives an example of the revised graph, in which nodes 2 and 3 are existing nodes and node 4 is a new order. We make dummy nodes 2', 3' and then build the meeting arc from $\{1, 2, 3\}$ to $\{2', 3', 4\}$. Nodes in the set $\{1, 2, 3\}$ or $\{2', 3', 4\}$ are directly connected by normal arcs. In this example, one feasible adjusted route for the truck is $1 - 2 - 3' - 4 - o$, which means that the truck starts from node 1, delivers to node 2, delivers to node 3, meets with a drone for the resupply of parcels for order 4 at node 3, and finally delivers to node 4 and returns to depot o .

EXAMPLE 3. Figure E.3 illustrates an example of this process. Consider a path sequence 1-2-3-4-5, where node 1 corresponds to the truck's starting position v_k . At Step i, we first set a truck's earliest available meeting time (marked as W_1 in the figure) to node 1 ($\tau_1 = W_1$). Then, we set a time schedule for the route following the rule $\tau_j = \max(W_j, \tau_i + T_{ij})$. We find that the service time of node 5 exceeds its hard time window based on this rule. To address this issue, we move node 5 to U_5^l and adjust the service times of nodes 3 and 4 correspondingly. The resulting initial time schedule is shown as Step ii.

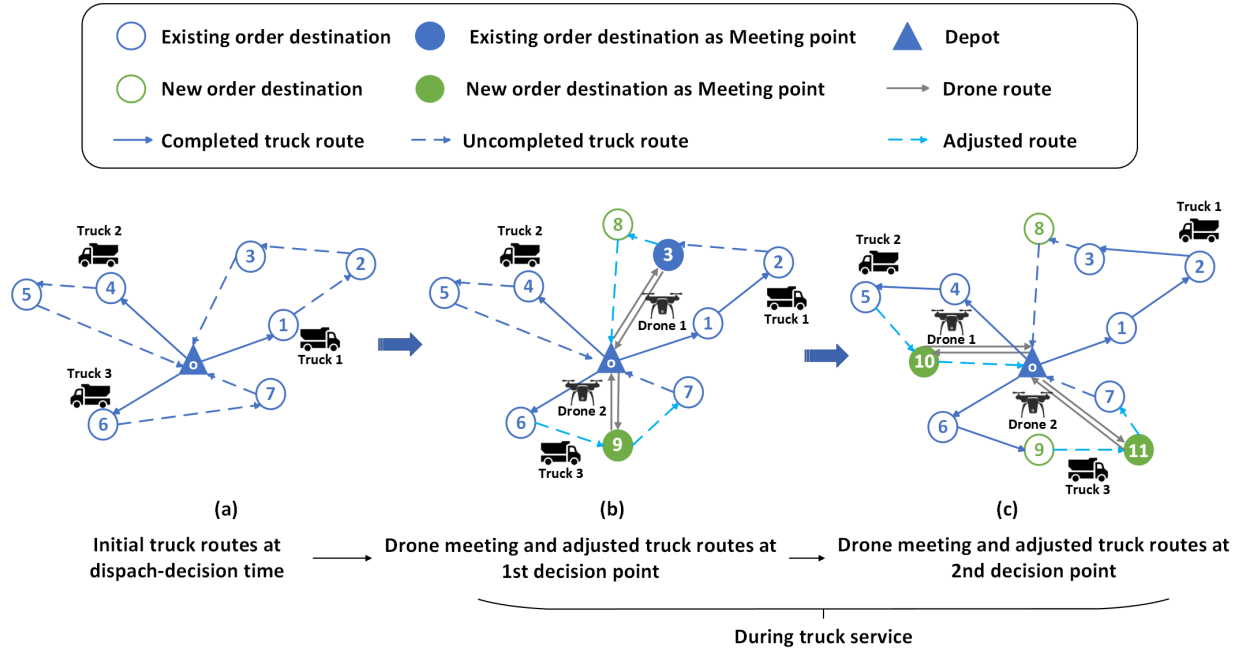


Figure E.1 Examples of meetings and routes.

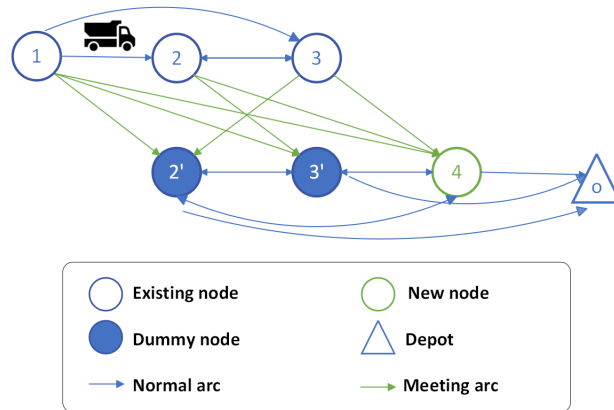
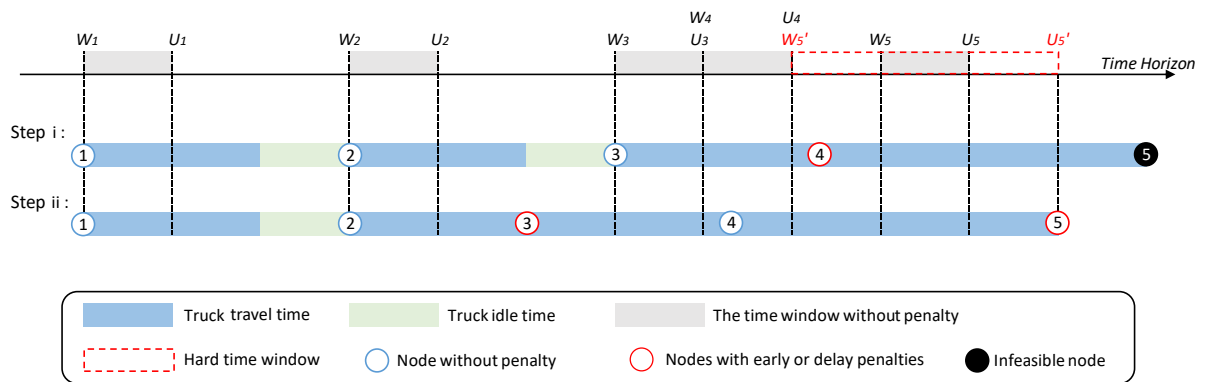
Figure E.2 Examples of revised graph G' with dummy nodes and meeting arcs.

Figure E.3 An example of initial time schedule construction.

EXAMPLE 4. Figure E.4 illustrates an example of a path that contains two segments. The first starts from node 7 and ends at node 5, and the second starts from node 4 and ends at node 1.

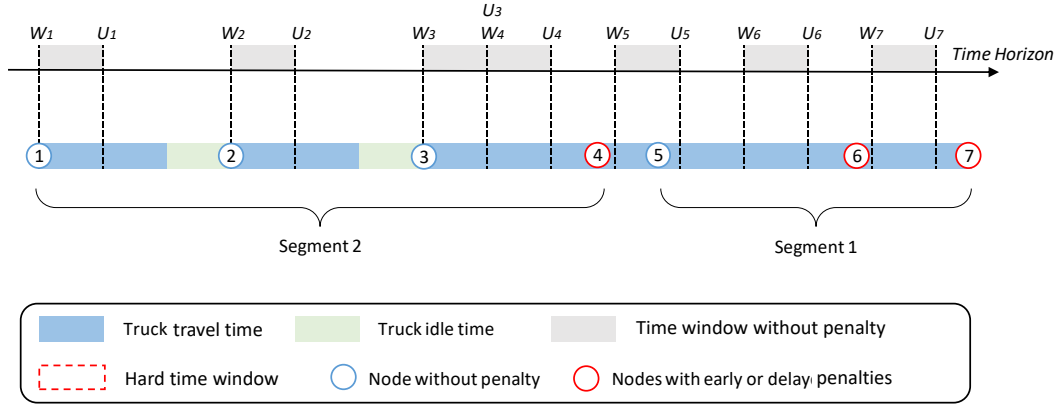


Figure E.4 An example of segments.

EXAMPLE 5. We revisit the two segments in Example 4. The first starts from node 7 and ends at node 5, and the second starts from node 4 and ends at node 1. Then, we analyze the two segments. For Segment 1, the ideal plan is to move the service time of nodes 7 and 6 to U_7 and U_6 , respectively, to eliminate delay penalties. However, if we move node 6 forward, the service time of node 5 also needs to be moved correspondingly, which results in penalties for early service on node 5. To determine the optimal movement, we need to test multiple movement schedules that reduce penalties. Specifically, we move forward the three nodes until node 7 reaches U_7 , which reduces the delay for nodes 6 and 7 without penalizing node 5. Then, further move nodes 5 and 6 until node 5 reaches W_5 , which reduces the delay penalty for node 6. Afterward, we can see that if we continue to move forward, the early penalty of node 5 will be incurred, which offsets the reduction of the delay penalty for node 6. Thus, the optimal movement for nodes 5 and 6 is $\tau_5 - W_5$, and that of node 7 is $\tau_7 - U_7$. Similarly, the optimal movement for Segment 2 is 0, because moving node 4 forward will cause node 3 to be served earlier than W_3 . Figure E.5 shows how the number of early and

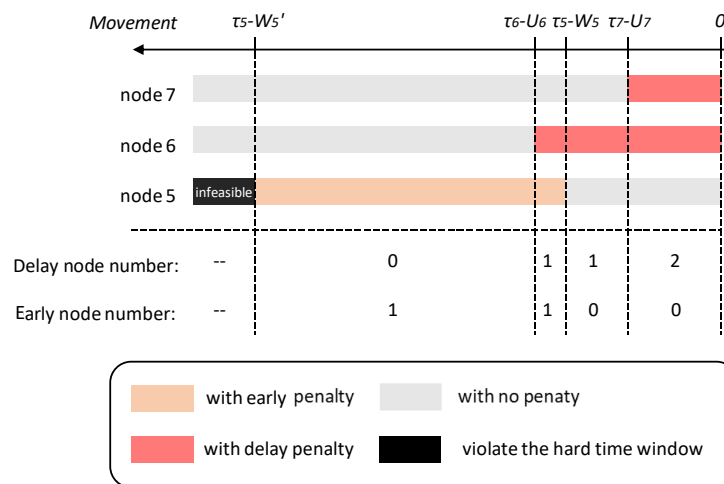


Figure E.5 Effects of moving forward Segment 1 on the number of early and delay nodes.

delay nodes changes as the segment moves forward, using Segment 1 as an example. The axis at the top of the figure represents the amount of the segment's forward movement. After moving forward by $\tau_5 - W_5$, the numbers of both early and delay nodes become 1, so this is the optimal movement for the segment.

EXAMPLE 6. Consider the path in Example 4. Since the optimal movement for node 5 is $\tau_5 - W_5$ and there is no truck idle time between nodes 4 and 5, node 4 should make the same movement as node 5 to ensure the feasibility of the time schedule. This conflicts with the optimal movement previously identified for node 4, which is 0. Therefore, when we consider the movement of Segment 1, we need to account for the corresponding movement needed for Segment 2. Such movement may affect the number of delay nodes and early nodes, and we should consider these effects when dealing with Segment 1. We incorporate Segment

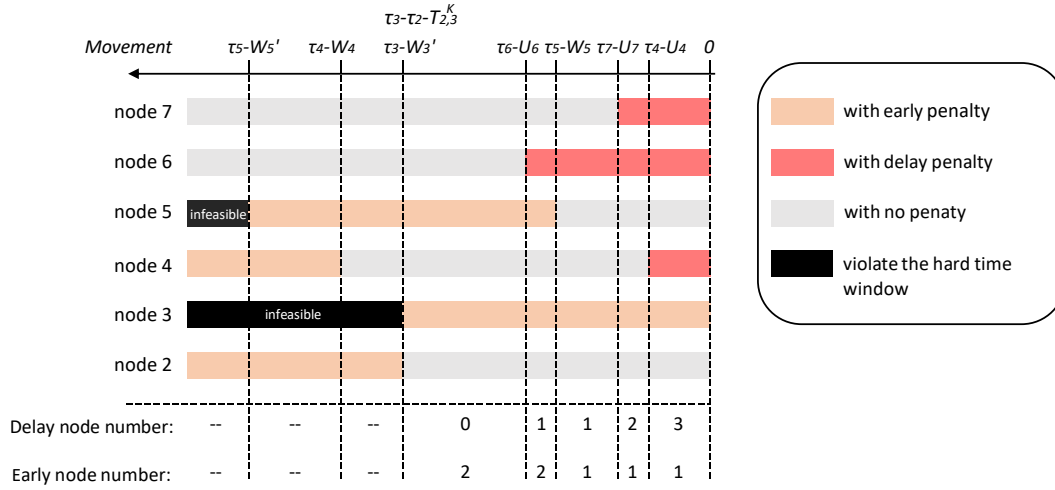


Figure E.6 Effects of moving forward two segments on the number of early and delay nodes.

2 in Segment 1 and process each node again sequentially from end to start. The process of the first three nodes (i.e., nodes 7, 6, 5) is the same as shown in Figure E.5. Next, the impact of the movement of the following four nodes (i.e. nodes 4, 3, 2, and 1) on the number of early and delay nodes is summarized in Figure E.6. Specifically, we can see that after moving $\tau_4 - U_4$, node 4 will not be penalized until it becomes an early node by moving $\tau_4 - W_4$. Node 3 will receive early penalties once it moves. Node 2 needs to move forward and receives penalties only when the movement of its subsequent nodes exceeds $\tau_3 - \tau_2 - T_{2,3}^K$, because there is a period of truck idle time between nodes 2 and 3. Similarly, if the movement of node 2 exceeds $\tau_2 - \tau_1 - T_{1,2}^K$, node 1 must also move. However, moving node 1 would result in an infeasible schedule, which imposes a restriction on the movement of node 2 such that it cannot exceed $\tau_2 - \tau_1 - T_{1,2}^K$.

In summary, with a forward movement of $\tau_7 - U_7$, the numbers of early and delay nodes are equal (both become 1). At this point, moving the nodes in segments 1 and 2 forward will not generate any additional benefits. We can thus conclude that these two segments have a combined optimal movement, which is $\tau_7 - U_7$.

F. Algorithms

F.1. Algorithm for Constructing Initial Time Schedule.

The algorithm for constructing the initial time schedule is shown in Algorithm 1.

F.2. Algorithm for Identifying the Optimal Movement.

The algorithm for identifying the optimal movement is shown in Algorithm 2.

Algorithm 1 Construct a feasible initial time schedule.

Input: Route r ; node j that violates constraint; current timetable $\tau^{current}$.

Output: New timetable: τ^{new} .

```

1: Initialize  $gap \leftarrow 0$ ;  $movement \leftarrow \tau_j^{current} - U_j'$ .
2: while the predecessor of  $j$  exists do
3:    $i \leftarrow$  the predecessor of  $j$ 
4:    $\tau_i^{new} \leftarrow \tau_i^{current} - movement$ 
5:    $gap \leftarrow \tau_j^{current} - \tau_i^{current} - T_{ij}$ 
6:    $movement \leftarrow movement - gap$ 
7:   if  $movement \leq 0$  then
8:     break
9:   end if
10:   $j \leftarrow i$ 
11: end while

```

G. Estimation of the Expected Incremental Costs for Postponed Services

First, we assess the incremental travel cost for trucks and drones that would be incurred if we postpone servicing the current orders (the order arrived at the current decision epoch and the orders that have been postponed at previous decision epochs). We begin by examining a specific current order m whose best assignment is truck k . We denote N as the total number of current orders that are assigned to truck k . In this scenario, the total incremental travel cost of truck k can be denoted as $F_m N$, where the average incremental travel cost per order is F_m . In assessing the incremental travel cost, we need to also involve the future information of the forthcoming order. For simplicity, we only focus on the next new order here, and we assume that each order arrives at a separate time point and is processed sequentially at its respective decision epoch. If the new order is assigned to a different truck from current order m , the incremental travel cost for serving each current order remains unchanged (denoted as F_m), since the new order is handled independently from the current order. However, if both the new and current orders are assigned to the same truck k , they can share the truck and drone routes, resulting in cost savings for each order. To summarize, the expected incremental travel cost for batch-servicing current order m with the next order can be expressed as

$$F^S = (1 - q)F_m + q \left(\frac{F_m N}{N + 1} \right), \quad (\text{G.1})$$

where q is the probability that the optimal truck assignment for the next order is the same as for the current order. In the scenario in which the geographic distribution of orders is random, this probability can be approximated as $1/|\mathcal{K}|$. q can also be estimated through simulation, given specific distribution data on orders.

Moreover, we also need to consider the potential rise in soft time-window penalties and the risk of failing to fulfill the current order due to postponement. Here, we denote the service time of immediately serving

Algorithm 2 Identify the optimal movement.**Input:** Segment: s ; node number in s : n ; delay node number in s : n^{delay} ; current timetable: $\tau^{current}$.**Output:** Optimal movement: *optimal_movement*.Initialize the number of early nodes: $n^{early} \leftarrow 0$; $gap \leftarrow 0$; $max_delay \leftarrow 0$.2: **for** $i \leftarrow s[n-1]$ to $s[0]$ **do** $m1 \leftarrow \tau_i^{current} - U_i$. $\triangleright$ check the time when i is moved to U_i .4: **if** $m1 > 0$ **then** $m1 \leftarrow m1 + gap$.6: Record that after moving $m1$, n^{delay} is decreased by 1.**end if**8: $m2 \leftarrow \max(0, \tau_i^{current} - W_i) + gap$ $\triangleright$ check the time when node i is moved to W_i .**if** $m2 \leq max_delay$ **then**10: Record that after moving $m2$, n^{early} is increased by 1.**else**12: Do not record because there is no need for i to move $m2$.**end if**14: **if** i is meeting point **then** $infeas_time \leftarrow \max(W'_i, T_{oi}^D)$ 16: **else** $infeas_time \leftarrow W'_i$ 18: **end if** $m3 \leftarrow \tau_i^{current} - infeas_time + gap$ $\triangleright$ check the time when node i is moved to infeasibility.20: **if** $m3 \leq max_delay$ **then**Record that the value of movement cannot exceed $m3$.22: **else**Do not record because there is no need for node i to move $m3$.24: **end if** $max_delay \leftarrow \max(max_delay, \tau_i^{current} - U_i)$.26: $j \leftarrow$ the predecessor of i .**if** j exists **then**28: $gap \leftarrow gap + \tau_i^{current} - \tau_j^{current} - T_{ji}^K$.**end if**30: **end for****for all** the movement that is being recorded **do**32: Check n^{early} and n^{delay} at that moment and find the *optimal_movement*.**end for**

this order as t_m , and the expected time interval between the arrivals of new orders as μ . We define the service time for the current order post-delay as $t_m + \phi \min(\mu, \delta)$, where ϕ is an impact coefficient that indicates the extent of delay in the final service time, initialized to 1, and δ is the self-triggered time interval, which is set to 1 hour in the implementation. If the post-delay service time $t_m + \phi \min(\mu, \delta)$ has already exceeded the latest completion time U'_m , immediate service is necessary; otherwise, we can estimate the soft time-window penalty for delayed services as $F^P = [(t_m + \phi \min(\mu, \delta)) - U_m]^+ F^{TW}$, where $[\cdot]^+$ denotes the positive part of $\cdot$ and F^{TW} is the time-window penalty of unit time.

By trading off the expected incremental costs for delayed services with the incremental cost of serving the order immediately, we can determine whether to postpone the order. Specifically, in our solution framework, we first set the temporary rejection cost of order F^R as the expected incremental costs for postponed services—i.e., $F^S + F^P$. Thus, if the incremental cost of immediate service exceeds F^R , the order will be postponed and handled at the next decision epoch. However, if the order remains unfulfilled after exceeding its maximum allowable postponement duration, it will be permanently rejected. In this case, the rejection penalty is set to the full outsourcing cost through third-party logistics providers.

H. Detailed Experimental Settings

Regarding the generated instances, we adopt the parameter settings from [Xia et al. \(2023\)](#), [Wang and Sheu \(2019\)](#). Specifically, the maximum drone capacity is set to 4 parcels; the speed of both trucks and drones is 40 km/h; the maximum drone flight duration is 30 minutes; and the unit travel costs for drones and trucks are \$0.083/min and \$0.021/min, respectively. Each instance contains a single depot, 16-72 existing orders, and 3-17 new orders. We name the instances as k_n_i (where k is the truck count, n is the node count, and i is the ID), with the number of drones consistent with trucks. Order destinations are uniformly distributed within 20 km, and the depot is located at the average coordinates of all nodes. For the time windows, the penalty-free latest delivery time U_i for order i is uniformly sampled from $[0.3, 0.15|\mathcal{N}|]$ hours. The penalty-free time window is constructed as $[W_i, U_i]$, where $W_i = \max(0, U_i - 0.5)$. Subsequently, the tolerable hard time window expands this interval to $[W'_i, U'_i] = [\max(0, W_i - 0.75), U_i + 0.75]$ by adding 45-minute buffers to both ends. To mimic the actual situation, we set the traveling distances of drones and trucks as the Euclidean distance and Manhattan distance, respectively.

For the real-world case study based on the Singapore dataset, we compile orders with pickup locations at the selected depots and release times during the evening rush hour (4:00 pm to 8:00 pm). We set a 50-minute time window for these orders, comprising a 10-minute desirable window and a 20-minute tolerable window before and after. Trucks are prearranged to return to the center and depart once an hour, while drones collaborate to fulfill newly arrived orders based on the proposed online policy. The vehicle and cost settings remain consistent with the synthetic instances. Finally, the maximum computational time for each algorithm call is limited to 5 seconds.

References

- Fachini RF, Armentano VA (2020) Exact and heuristic dynamic programming algorithms for the traveling salesman problem with flexible time windows. *Optimization Letters* 14:579–609.
- Li JQ (2010) A computational study of bi-directional dynamic programming for the traveling salesman problem with time windows. <https://api.semanticscholar.org/CorpusID:9784274>.
- Wang Z, Sheu JB (2019) Vehicle routing problem with drones. *Transportation Research Part B: Methodological* 122:350–364.
- Xia Y, Zeng W, Zhang C, Yang H (2023) A branch-and-price-and-cut algorithm for the vehicle routing problem with load-dependent drones. *Transportation Research Part B: Methodological* 171:80–110.